

Figure 2: Procedural console applications in Vala

Vala voilà!

I certainly can't cover every detail of Vala—that would need a big reference book. I'll show you some of its notable features, which are comparable to C, C++, C#, Java, etc. My main aim in this section is to show how to build high-performance applications targeting different functionalities of GNOME, and to establish a necessary foundation for the later section on Genie. I encourage you to refer to the Vala links provided in the *References* section, to explore it in depth.

```
int main()
{
    stdout.printf("\n Hello to FLOSS from Vala!!!\n");
    return 0;
}
```

To see Vala in action, compile *hellovala.vala* (source shown above, look for the source folder on the magazine CD) with the command *valac hellovala.vala*. You can clearly see the resemblance of Vala code to C code. Run the compiled binary with *./hellovala*—by default, *valac* names the executable after the Vala source file, so don't look for *a.out*! You can change the output file name with the -o option. Also, you can make *valac* generate only the intermediate C source file, with the *-C* option. This file contains GObject-based C code, which may be interesting to read.

Vala provides various basic data types, which are grouped into:

- types whose widths depend upon the underlying platform, as in C, C++, etc.
- integer types whose width is independent of the underlying platform, as in C#, Java, etc.

Vala also provides ways to learn about the maximum and minimum values represented by various data types. It provides 'built-in' objects *stdin* and *stdout* for input and the console output.

Vala provides the *string* data type to deal with immutable strings. If you want to work with mutable strings, then you have to instantiate the *StringBuilder* class, which has methods like *append*, *prepend*, and *insert* for mutable strings. Vala also provides a *string template* feature—by prefixing strings with an @ operator, this feature can evaluate embedded variables and expressions prefixed with $. Vala also provides *verbatim strings* that keep *escape sequences*, *line breaks* and *quotation marks*, as is, without any special interpretation. Verbatim strings are contained in *triple quotes* (" " ").

Vala has a *type interface* mechanism that lets you define a local variable using the *var* keyword instead of a data type, if the type of the variable is unambiguous (by the initial value assigned to it). Compile and run *datatypes.vala* from the CD to see data types, string types and the type interface feature in action.

In *datatypes.vala*, *main* returns void so the return value will be 0 when *main* completes execution. The /** */ style comments in the above program are not special to *valac*, but the Vala documentation generation utility, *valadoc*, treats those in special ways. To read more about *valadoc*, see the *References* section. The output of *datatypes.vala* is shown in Figure 2.

If you want to use various collection objects like lists, hash maps, sets and so on, then you have to build and install *libgee*, since those are not currently provided by Vala. Download the latest *libgee* source tarball from the *libgee* homepage (see *References*), and run the following commands:

```
tar -jxvf libgee-version.tar.bz2 && cd libgee-version
./configure && make && sudo make install
```

To see Vala collections in action, compile the program shown below with *valac --pkg=gee-1.0 collections.vala* and execute it:

```
using Gee;

void main() {
        var list = new ArrayList<int> ();
        var set  = new HashSet<int> ();

        stdout.printf("\n");
        for(var i=1; i <= 10; ++i) {
          var iVal = 0;
          stdout.printf(" Enter value %u : ", i);
          stdin.scanf("%d", out iVal);
          list.add(iVal);
        }

    foreach(var j in list) {
        set.add(j);
    }

    stdout.printf("\n Values after eliminating the duplicates =>\n");
    foreach(var k in set) {
        stdout.printf(" %d\n", k);
    }
    stdout.printf("\n");
}
```

Note the similarity in the above code to C++ containers